
Virtual Analog Neural Audio Modeling

End-to-End Deep Learning Pipeline for Real-Time DSP and VST3 Audio Plugins

Abstract

This document details the engineering specifications, mathematical formulations, and execution procedures for capturing, training, and deploying virtual analog neural audio models. The architecture employs single-layer stateful recurrent neural networks (GRU or LSTM) combined with multi-domain loss optimization (batch-normalized Error-to-Signal Ratio, first-order FIR pre-emphasis, and multi-resolution STFT) to model nonlinear analog saturation, dynamic envelope compression, and harmonic distortion with sub-millisecond real-time execution inside standard Digital Audio Workstation (DAW) audio threads.

1 System Architecture and Real-Time Execution Constraints

Analog audio processing equipment—including vacuum tube preamplifiers, magnetic tape machines, diode clipping stages, and optoelectronic compressors—operates as continuous, stateful, nonlinear dynamic systems. Modeling these systems in digital software requires balancing acoustic waveform fidelity against the strict deterministic time budgets of real-time audio threads.

Real-Time DAW Thread Constraints:

In a professional DAW environment (running at a 44.1 kHz or 48 kHz sample rate with a 64-sample buffer), the audio callback thread must process 64 consecutive samples in under 1.45 ms. Any computational delay exceeding this window triggers an operating system audio buffer underrun, causing audible digital crackles and dropouts. Consequently, virtual analog neural networks must satisfy three architectural rules:

- Zero Algorithmic Latency:** Forward evaluation must operate on instantaneous audio samples $x[n]$ without future lookahead windows.
- L1 CPU Cache Locality:** The network parameter footprint and recurrent state buffers must reside entirely within high-speed CPU L1 cache (32 KB to 64 KB) to prevent memory bus wait states.
- Deterministic CPU Budget:** Total inference time per audio block must consume under 1.0% of a single CPU core.

1.1 Recurrent Formulation vs. Alternative Topologies

Modern deep learning architectures exhibit contrasting trade-offs when applied to sample-by-sample real-time audio:

- **Feedforward Multi-Layer Perceptrons (MLPs):** Possess zero internal memory states. While computationally lightweight, MLPs are static memoryless functions $y[n] = f(x[n])$. They cannot model frequency-dependent impedance, power supply sag, or dynamic time constants.
- **WaveNet / Dilated Convolutional Networks (TCNs):** Causal dilated convolutions model temporal memory through finite receptive fields. However, real-time evaluation requires circular history buffers and multiple stacked dilation layers, consuming $5\times$ to $10\times$ more CPU than compact recurrent cells.
- **Transformers and Self-Attention:** Attention matrices scale quadratically with sequence history ($\mathcal{O}(L^2)$) and require future or block lookahead, making them unsuitable for sample-by-sample sub-millisecond audio streaming.
- **Gated Recurrent Networks (GRU / LSTM):** Maintain internal continuous state vectors $\mathbf{h}[n]$ updated sample-by-sample: $\mathbf{h}[n] = f(x[n], \mathbf{h}[n-1])$. This precisely maps to physical analog state variables (capacitor voltages and transformer magnetic flux), providing infinite impulse response (IIR) memory with constant $\mathcal{O}(1)$ computation per sample.

2 Recurrent Neural Architecture Design

The neural core in this template is designed as a streamlined two-stage pipeline:

$$\mathbf{x}[n] \in \mathbb{R}^{1+P} \xrightarrow{\text{Stage 1}} \text{Recurrent Cell (GRU or LSTM)} \xrightarrow{\text{Stage 2}} \text{Linear Readout} \rightarrow y[n] \in \mathbb{R} \quad (1)$$

where P is the number of continuous hardware conditioning parameters (such as Drive or Tone).

2.1 Recurrent Cell Selection: GRU vs. LSTM

- **Gated Recurrent Unit (GRU, Recommended Default):**

Features 3 internal gates (reset gate $\mathbf{r}$, update gate $\mathbf{z}$, and candidate state $\mathbf{n}$) and a single hidden state vector $\mathbf{h} \in \mathbb{R}^H$. GRU requires 25% fewer floating-point operations than LSTM, drawing under 0.8% CPU per core. It is the optimal model for circuits with continuous state memory and uniform exponential decay: vacuum tube stages, clipping diodes, magnetic tape saturation, solid-state preamps, and guitar amplifiers.

- **Long Short-Term Memory (LSTM):**

Features 4 gates (input, forget, output, and candidate) with decoupled cell state $\mathbf{c} \in \mathbb{R}^H$ and hidden state $\mathbf{h} \in \mathbb{R}^H$. It requires $\sim 25\%$ higher CPU consumption. LSTM is strictly necessary when modeling circuits with multi-rate physical time constants that span multiple orders of magnitude. For example, optical compressors (such as the Teletronix LA-2A) use cadmium sulfide photoresistors exhibiting instantaneous sub-millisecond attack coupled with multi-second non-exponential release curves. The separate cell state $\mathbf{c}$ maintains long-term charge integration without premature exponential decay.

2.2 Hidden State Dimensioning

The recurrent hidden dimension H controls the number of internal state variables and non-linear expressive capacity:

Configuration	Hidden Size (H)	Parameters	Target Circuit Suitability
Compact	32	$\sim 3,500$	Clean boost, console line amps, transparent master utility ($< 0.4\%$ CPU).
Standard	48	$\sim 7,500$	Vacuum tube stages, tape machines, overdrive/distortion ($< 0.8\%$ CPU).
High Capacity	64	$\sim 13,000$	Hard diode clipping, multi-stage metal lead channels, extreme square-wave fuzz.

Table 1: Recurrent cell sizing and hardware allocation benchmarks.

Single-Layer Execution Constraint ($N_{\text{layers}} = 1$):

While multi-layer recurrent networks are prevalent in general deep learning, stacking recurrent layers in real-time virtual analog plugins is counterproductive:

1. **L1 Cache Preservation:** A single 48-unit GRU contains exactly 7,537 32-bit floating-point parameters (~ 30.1 KB), fitting comfortably inside a 32 KB L1 cache. Stacking a second layer doubles parameter memory and introduces inter-layer pipeline stalls.
2. **Empirical Diminishing Returns:** Academic virtual analog benchmarks (Wright & Välimäki, JAES 2020) demonstrate that stacking 2 or 3 recurrent layers improves Error-to-Signal Ratio by less than 0.05% while doubling CPU execution overhead.

2.3 Residual Connection Mechanics

The residual connection allows audio to bypass the recurrent network:

$$y[n] = \begin{cases} \text{Net}(x[n]) + x_{\text{dry}}[n], & \text{if residual connection is active} \\ \text{Net}(x[n]), & \text{if residual connection is disabled} \end{cases} \quad (2)$$

When to Set Residual Connection Flag:

- **Set `use_residual`: `true` for Clean Linear Amplification:**
Applies to transparent line boosters and clean console preamps where the output is 90%+ identical to the input. Bypassing the linear component allows the network to dedicate 100% of its capacity to modeling subtle harmonic coloration.
- **Set `use_residual`: `false` for Distortion, Tape, Fuzz, and Amps:**
Distortion and clipping circuits fundamentally alter waveform symmetry and phase response. Summing the original clean dry signal back into the distorted output causes destructive phase cancellation, thinning out low-frequency punch and introducing unnatural comb filtering.

3 Procedural Audio Excitation and Gain Calibration

Training a generalizable neural model requires exciting every state variable across amplitude, frequency, and slew-rate dimensions without introducing acoustic bias.

3.1 Procedural Signal Composition

- **Logarithmic Chirp Sweeps (35s, -30 to -1 dBFS):** Excites the continuous spectrum from 20 Hz to 20 kHz across four discrete amplitude levels to capture frequency response and level-dependent harmonic generation.
- **Synthesized Drum Suite (55s):** High-amplitude transient bursts (> 16 dB crest factor) with sub-millisecond rise times to probe dynamic peak clamping and slew-rate limits.
- **Incommensurate Multitones (35s):** Prime-frequency sinusoidal combinations (53 Hz, 107 Hz, 223 Hz, 439 Hz) sharing no common harmonic multiples. Any new frequency components appearing in the output correspond strictly to intermodulation distortion (IMD).
- **Variable-Slew Impulses (25s):** Pulses with rise times from 0.1 ms to 10 ms to evaluate circuit recovery from abrupt voltage transitions.

3.2 Sample Latency Alignment via Synchronization Pip

Analog-to-digital and digital-to-analog converters introduce input/output buffer latency (typically 1 to 500 samples). If the recorded target signal $y[n]$ is shifted relative to dry input $x[n]$ by even a single sample, gradient descent forces the recurrent network to act as a digital delay line, burning internal neurons on phase compensation.

Alignment Procedure via 1-Sample Sync Pip:

The excitation track `dry_corpus.wav` contains 0.5 seconds of digital silence followed by a **single-sample unit impulse at exactly $t = 0.500$ s (sample 22,050)**:

1. Open the DAW project and zoom into the wet track at sample 22,050.
2. Locate the sharp vertical impulse.
3. Nudge the wet recording horizontally until the wet impulse aligns sample-accurately with the dry impulse.

3.3 Master Pad Calibration Steps

High-drive saturation naturally compresses audio, increasing average signal energy. To prevent digital clipping on hot recordings:

1. Record all parameter passes (e.g. Drive 0.0, 0.5, 1.0) with fixed hardware analog gain.
2. Solo the maximum drive pass (Drive = 1.0) and inspect the DAW master true-peak meter.
3. If the peak reads $+3.2$ dBFS, lower the DAW master output fader by 4.2 dB so the peak lands at -1.0 dBFS.
4. Export all passes through that exact fader setting, preserving 1.0 dB of digital headroom and 100% of the dynamic range across all passes.

4 Dual-Domain Compound Loss Formulation

Standard Mean Squared Error (MSE) produces dull, muffled models when applied to audio because low frequencies dominate raw voltage energy while high-frequency overtones have minute physical amplitudes. The pipeline resolves this using a composite dual-domain loss:

$$\mathcal{L}_{\text{total}} = w_{\text{esr}} \cdot \mathcal{L}_{\text{ESR}} + w_{\text{stft}} \cdot \mathcal{L}_{\text{STFT}} + w_{\text{dc}} \cdot \mathcal{L}_{\text{DC}} \quad (3)$$

4.1 Batch-Wide Error-to-Signal Ratio ($\mathcal{L}_{\text{ESR}}$)

Calculates normalized waveform error across the mini-batch to prevent denominator collapse during quiet intervals:

$$\mathcal{L}_{\text{ESR}} = \frac{\sum_{b=1}^B \sum_{t=1}^L (y_{\text{true}}[b, t] - y_{\text{pred}}[b, t])^2}{\sum_{b=1}^B \sum_{t=1}^L y_{\text{true}}[b, t]^2 + \epsilon} \quad (4)$$

Prior to computing $\mathcal{L}_{\text{ESR}}$, both prediction and target pass through a first-order FIR pre-emphasis high-pass filter:

$$\tilde{y}[n] = y[n] - \alpha \cdot y[n-1], \quad \alpha = 0.85 \quad (5)$$

This attenuates overpowering sub-bass energy and amplifies high-frequency error gradients, forcing the network to accurately track subtle clipping edges.

4.2 Multi-Resolution Spectral Loss ($\mathcal{L}_{\text{STFT}}$)

To ensure tonal balance independently of microscopic sub-millisecond phase shifts, Short-Time Fourier Transforms are computed across three window resolutions ($N = 512, 1024, 2048$):

$$\mathcal{L}_{\text{STFT}} = \frac{1}{3} \sum_{i=1}^3 (\mathcal{L}_{\text{sc}}^{(i)} + \mathcal{L}_{\text{mag}}^{(i)}) \quad (6)$$

where $\mathcal{L}_{\text{sc}}$ is spectral convergence and $\mathcal{L}_{\text{mag}}$ is log-magnitude distance clamped to a -70 dBFS floor to ignore background converter noise:

$$\mathcal{L}_{\text{sc}} = \frac{\| |Y| - |\hat{Y}| \|_F}{\| |Y| \|_F}, \quad \mathcal{L}_{\text{mag}} = \frac{1}{M} \|\log(|Y| + \epsilon) - \log(|\hat{Y}| + \epsilon)\|_1 \quad (7)$$

4.3 Transient Burn-In Window

To prevent cold-start boundary click penalties, the first 128 samples (~ 2.9 ms) of each 2048-sample chunk are omitted from backpropagation. This permits internal recurrent states to settle into dynamic equilibrium before computing gradients.

5 Training Execution and Telemetry Verification

5.1 Training Hardware Runtimes (30 Epochs)

- **NVIDIA CUDA GPU (RTX 3060+):** ~ 5 minutes.
- **Apple Silicon (M1/M2/M3 via MPS):** ~ 8 minutes.
- **Standard Desktop CPU (Intel i5/i7 / AMD Ryzen):** ~ 15 to 20 minutes (~ 35 s per epoch).

5.2 Epoch Telemetry Dictionary

During training, `train.py` outputs a real-time terminal telemetry string:

```
Epoch 28/40 | Train: 0.0182 | Val: 0.0214 | ESR: 0.0152 (1.52%) | STFT: 0.0410 | DC
: 0.0001 | LR: 1.25e-04
```

Token	Value	Physical Meaning and Quality Threshold
Train	0.0182	Compound objective loss across training split.
Val	0.0214	Compound objective loss across validation split.
ESR	0.0152	Waveform error percentage (1.52%). < 3% denotes double-blind transparent fidelity.
STFT	0.0410	Spectral distance. < 0.08 denotes indistinguishable tonal and harmonic balance.
DC	0.0001	Centerline voltage drift. < 0.001 confirms zero clicks or thumps.
LR	1.25e-04	Scheduled learning rate, decaying via Cosine Annealing to 1.0e-5.

Table 2: Training telemetry token breakdown and numerical target criteria.

6 Evaluation Protocols and Systematic Architecture Tuning

Following training, `evaluate.py` runs stateful sequential inference across all parameter positions, streaming audio through `weights_best.pt` and generating an objective verification report:

Knob Setting	ESR (%)	STFT Dist	Peak Model	Peak Target	Peak Delta	Verdict
Drive = 0.00	1.14%	0.0412	-6.02 dBFS	-6.00 dBFS	-0.02 dB	Excellent (Clean Match)
Drive = 0.25	1.89%	0.0520	-4.10 dBFS	-4.08 dBFS	-0.02 dB	Excellent (Edge of Breakup)
Drive = 0.50	2.65%	0.0684	-2.45 dBFS	-2.40 dBFS	-0.05 dB	Excellent (Harmonic Saturation)
Drive = 0.75	3.42%	0.0811	-1.20 dBFS	-1.15 dBFS	-0.05 dB	Good (Heavy Saturation)
Drive = 1.00	4.10%	0.0935	-0.45 dBFS	-0.40 dBFS	-0.05 dB	Good (Full Hard Clipping)

Table 3: Annotated model evaluation metrics across physical parameter sweep.

6.1 Systematic Architecture Adjustment Decision Tree

When evaluation telemetry or listening tests reveal modeling inaccuracies, use this deterministic decision matrix to adjust parameters directly in `training/config.json`:

Observed Symptom	Physical Root Cause	Prescribed Action in <code>config.json</code>
High ESR ($> 8.0\%$) across all knob settings	Latency misalignment or incorrect residual setting.	1. Check sync pip in DAW at sample 22,050. 2. Set <code>"use_residual": false</code> for distortion/tape.
Low ESR ($< 3.0\%$) but muffled highs and missing bite	Waveform loss dominated by bass; treble underweighted.	Increase spectral weight: <code>"stft_weight": 0.35</code> and pre-emphasis: <code>"pre_emphasis_coeff": 0.92</code> .
Rounded clipping corners on square-wave fuzz	Insufficient recurrent capacity for hard discontinuities.	Increase hidden size: <code>"hidden_size": 64</code> and re-train for 30 epochs.
Accurate at min/max, but inaccurate at middle (0.5)	Non-linear logarithmic potentiometer taper.	Re-record hardware with 11 discrete steps (0.0, 0.1, ..., 1.0) concentrated at the saturation knee.
Val loss diverges after epoch 20 (overfitting)	Network memorizing synthetic corpus patterns.	Automated export uses <code>weights_best.pt</code> ; reduce <code>"hidden_size": 32</code> or lengthen training audio.

Table 4: Systematic hyperparameter remediation decision tree.

7 Weight Export and Embedded C++ Deployment

The trained checkpoint (`weights_best.pt`) cannot be executed inside a DAW via Python. The template compiles models directly into real-time C++ using **RTNeural** and **JUCE 8**.

7.1 Automated Weight Transformation (`export_rtneural.py`)

- Gate Reordering:** PyTorch stores GRU gates as `[reset, update, new]` (r, z, n). RTNeural requires `[update, reset, hidden]` (z, r, h).
- Matrix Transposition:** Weight matrices are mapped to row-major format for SIMD vectorized execution.
- Bias Merging:** Input and hidden bias vectors are summed into a single bias vector per gate.
- Numerical Verification:** Runs forward inference comparing PyTorch against manual C++ step arithmetic, asserting absolute discrepancy below 10^{-6} .
- Automated Header Embedding:** Writes formatted JSON directly into `plugin_template/Source/EmbeddedModel.h` as a C++ raw string literal.

7.2 C++ Real-Time Audio Callback Integration

In `PluginProcessor.cpp`, audio samples and continuous knob parameters are evaluated sample-by-sample:

```
// Process sample-by-sample in DAW audio callback thread
float inputSample[2] = { dryAudioSample, *driveParameter };
float wetAudioSample = engine.processSample(inputSample);
channelData[sample] = wetAudioSample;
```

7.3 Compilation Commands

```
cmake -B build -S plugin_template  
cmake --build build --config Release
```

The resulting VST3 binary is written to `build/GenericNeuralAudioPlugin_artefacts/Release/VST3/`.